

National University of Singapore
School of Computing

Semester 1, AY2022-23

CS4246/CS5446

AI Planning and Decision Making

Issued: 8 Sep 2022

Due: 26 Sep 2022@23:59

Assignment 2

General Guidelines

Please complete the Assignment in the **same 2-person TEAM** as for past assignment(s). **ONE person from each team should submit ALL the answers (for both programming and homework problems)**. Grading will be done team-wise.

On collaboration

- You are encouraged to discuss solution ideas. However, each team *must write up the solutions independently*. It is considered as plagiarism if the solution write-up is highly similar to other teams's write-ups or to other sources.

Homework Problems Submission

- Submission will take place through **LumiNUS Quiz**.
- **No late submissions for the homework assignment solutions are allowed.**

Programming Problems Submission

- **ONE person from each team should submit the solutions.** Grading will be done team-wise.
- **IMPORTANT!** Please write the following on the top of your solution files for EACH problem in the assignment:
 - Name, metric number and Tutorial Group of **all team members** as they appear on LumiNUS
 - Collaborators (write **None** if no collaborators)
 - Sources, if any, or if you obtained the solutions through research, e.g. through the web.
- **Upload the code of your programming assignment to the aiVLE evaluation server**, by following the instructions [here](#). Please keep a copy of your submission zip file for safety.
- Please name your files, using ONE of the members' information according to this format:
A2_<Member1-TutorialGroup>-<Member1-name>-Task
E.g., A2-TG6-xiaomei.tan-Task.zip
- **A late score penalty of 20% will be incurred for late programming assignment solutions submissions.**

1 Homework Assignment (LumiNUS Quiz)

[Preparation]: For this problem, while the numbers may be worked out manually, you are encouraged to use one of the academic or free trial versions of computer programs for decision modeling, e.g., [Bayesfusion GeNIE modeler](#) or [SMILE engine](#), [Netica](#), [Syncopian DPL 9](#), [TreeAge](#), etc., to manipulate the influence diagrams and the decision trees.

1.1 Homework Problem 1: Decision Theory

[10 marks]

As the Mid-Autumn Festival draws near, little Curious Bean has just found a delicious-looking mooncake in the fridge. She is concerned that the mooncake contains Durian, which she really dislikes but she still wants to eat the mooncake.

If she eats the mooncake and it contains Durian, she will receive a utility of -100 and if the mooncake doesn't contain Durian, she will receive a utility of 10. If she doesn't eat the mooncake she will get 0 utility. Being pink in color, there is a probability of 0.1 that the mooncake contains Durian.

a) Should Curious eat the mooncake? Why or why not?

Curious can now smell the mooncake to judge whether it contains Durian before she eats it. However, since she has a runny nose and cannot smell well, she will incorrectly identify Durian when there is No Durian with probability 0.2 and will incorrectly identify No Durian when there is Durian with probability 0.3.

b) Represent Curious' decision problem in (i) a decision network and (ii) a decision tree, using the following variables: E is her choice to eat, U is her utility earned, D is whether the mooncake contains Durian, and S is her attempt at smelling.

Curious has just smelled the mooncake and she thinks it doesn't have Durian.

c) Write the probability, X, that the mooncake has Durian given that she smelled No Durian? (Answer with two decimal places).

d) What is her maximum expected utility, Y given that she smelled No Durian?

e) What is the Value of Perfect Information (VPI) of smelling the Durian?

[Give all answers as a decimal (number), or simplest form fraction in terms of the relevant variables X and Y.]

1.2 Homework Problem 2: Markov Decision Process

[10 marks]

Consider an **undiscounted** MDP having three states, (1, 2, 3), with rewards $-1, -2, 0$, respectively. State 3 is a terminal state. In states 1 and 2 there are two possible actions: a and b . The transition model is as follows:

- In state 1, action a moves the agent to state 2 with probability 0.7 and makes the agent stay in original state with probability 0.3.

- In state 2, action a moves the agent to state 1 with probability 0.7 and makes the agent stay in original state with probability 0.3.
 - In either state 1 or state 2, action b moves the agent to state 3 with probability 0.2 and makes the agent stay in original state with probability 0.8.
- a) Draw the state transition diagram for action a and b respectively.
 - b) What can be determined qualitatively about the optimal policy in states 1 and 2?
 - c) Apply policy iteration, showing each step in full, to determine the optimal policy and the values of state 1 and 2. Assume that the initial policy has action b in both states.
 - d) What happens to policy iteration if the initial policy has action a in both states? Does discounting help? Does the optimal policy depend on the discount factor?
-

2 Programming Assignment (aiVLE submission)

In the first assignment, we learnt to solve deterministic planning problems with PDDL solvers. In this Assignment, we will learn to solve planning problems in non-deterministic and stochastic environments with two separate algorithms, Value Iteration and Monte Carlo Tree Search.

Installation Instructions

We assume that you have already set up the programming environment for the assignments.

- Setup docker on your machine, instructions for which can be found [here](#).
- Pull the docker that we have already setup for you with all the required dependencies. You can follow the instructions [here](#) to do so.
- After pulling the docker image, test your your installation by running
docker run -it --rm -v \$PWD:/workspace cs4246/base python test_installation.py
for Linux/Mac or
docker run -it --rm -v %(cd)\%:/workspace cs4246/base
python test_installation.py for Windows (file given along with this assignment) on the docker container.

Programming Assignment Submission Instructions

Please follow the instructions listed [here](#). Task specific submission instructions are as follows:

- a. For Task 1: You have been provided with a vi_agent_template.zip file in the correct submission format. Complete the functions marked with “FILL ME” in __init__.py. Your submission zip file should have the exact same structure as the zip file you received.
- b. For Task 2: Modify the __init__.py in the mcts_agent_template.zip by completing the functions with “FILL ME” and make a separate submission.

Note :

1. run the __init__.py script with an argument N where N=0,1,2,3,4,5 for each task to test your codes.
2. Please do not print anything to the console, as it might interfere with the grading script.

Getting started

We will be using the same gym_grid_environment (<https://github.com/cs4246/gym-grid-driving>) to simulate the solutions. All dependencies have been installed in the docker image “cs4246/base” which you have already downloaded for Assignment 1.

By now, you should be familiar with the gym_grid_environment. Go through the IPython Notebook file on Google Colab [here](#), if you have already not done so in the previous programming assignment.

You are now ready to begin solving the two tasks which are listed below!

2.1 Programming problem 1: Value Iteration

[10 marks] In the crossing the road task in Assignment 1, we assumed all the cars move with a constant speed of -1 . Most of the times, this assumption does not hold. There is inherent noise in the car speeds due to reasons like bumpy roads, uneven driving, etc. This non-deterministic behaviour is not captured in the PDDL format.

For problems that are not very large, planning algorithms like Value Iteration can be used to handle non-deterministic behaviours. In this task, we model the problem as a Markov Decision Process and use Value Iteration algorithm to find the optimal policy.

We have provided the script which models the problem as an MDP. Every state in this MDP is represented as a tuple of features: $\{\text{Agent}_x, \text{Agent}_y, \text{Car}_x^1, \text{Car}_y^1, \dots, \text{Car}_x^N, \text{Car}_y^N, \text{isTerminal}\}$, where:

- $\text{Agent}_x, \text{Agent}_y$ denote the x and y coordinates agent.
- $\text{Car}_x^N, \text{Car}_y^N$ denote the x and y coordinates of Car^N .
- isTerminal is a boolean variable denoting whether the state is terminal.

Your task is to fill up the function that implements the Value Iteration algorithm which is marked with “FILL ME”.

You can test your code with the test configurations given in the script by running `python __init__.py N`, where N is an int value between 0 to 5, on the docker (using the `docker run ...` command).

HINT : You can use Matrix multiplication to optimise the code, but refrain from using the function `np.matmul()/np.multiply()/np.dot()` as these functions interfere with the evaluation script. Instead you can use function `matmul()` defined in the same script.

2.2 Programming problem 2: Monte Carlo Tree Search

[20 marks] Unfortunately, we moved to a bigger parking lot and Value Iteration is not feasible as it does not scale well with large state spaces. However, we can use Monte Carlo Tree Search (MCTS) to handle such cases.

We have provided a separate script which solves the problem with MCTS, Your task is to complete the script by filling up 2 code snippets inside the functions marked with “FILL ME”.

The functions required to be filled up are:

- `backpropagate()` : This function should implement the backpropagation step of MCTS.
- `chooseBestActionNode()` : Populate the list `bestNodes` with all children having maximum value. The value of the i^{th} child node can be calculated with the formula:

$$\frac{v_i}{n_i} + c\sqrt{\frac{\log N}{n_i}}$$

where,

- v_i : sum of returns from the i^{th} child node
- n_i : Number of visits of the i^{th} child node
- N : Number of visits of the current node
- c : The exploration constant. We set it to be 1.

For more details on MCTS, it might be helpful to look [here](#). You are also encouraged to look through the rest of the code to see how the entire MCTS algorithm is implemented.

You can test your code with the test configurations given in the script by running `python __init__.py N`, where N is an int value between 0 to 5, on the docker (using the `docker run ...` command).
